

**КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА**

Факультет комп'ютерних наук та кібернетики

Кафедра інтелектуальних програмних систем

Курсова робота

за спеціальністю 121 Інженерія програмного забезпечення

на тему:

**РОЗРОБКА ІНТЕРАКТИВНОГО ВЕБСЕРЕДОВИЩА ДЛЯ
МОДЕЛЮВАННЯ ТА ВІЗУАЛІЗАЦІЇ СИСТЕМ НЕЛІНІЙНОЇ
ДИНАМІКИ**

Виконала студентка 3-го курсу
ОПП «Програмна інженерія»
Челушкіна Валерія Олександрівна

(підпис)

Науковий керівник:
доцент кафедри інтелектуальних програмних систем
кандидат фіз.-мат. наук, доцент
Галкін Олександр Володимирович

(підпис)

Засвідчую, що в цій роботі
немає запозичень з праць інших авторів
без відповідних посилань.

Студентка

(підпис)

РЕФЕРАТ

Обсяг роботи 38 сторінок, 7 ілюстрацій, 1 графік, 2 таблиці, 21 джерело посилань, 5 додатків.

НЕЛІНІЙНА ДИНАМІКА, ТЕОРІЯ ХАОСУ, АТРАКТОРИ, ІНТЕРАКТИВНЕ ВЕБСЕРЕДОВИЩЕ, ВІЗУАЛІЗАЦІЯ, ЧИСЕЛЬНЕ ІНТЕГРУВАННЯ, ОПТИМІЗАЦІЯ, ПРОДУКТИВНІСТЬ.

Об'єктом дослідження є процеси чисельного моделювання та візуалізації складних хаотичних динамічних систем у реальному часі.

Метою роботи є розробка та покращення ефективності інтерактивного вебсередовища для моделювання та візуалізації систем нелінійної динаміки.

Методи розроблення: чисельне інтегрування систем звичайних диференціальних рівнянь методом Рунге-Кутти 4-го порядку, методи комп'ютерної графіки реального часу на базі WebGL, швидке управління станом та реляційне моделювання даних.

Інструменти розроблення: мови програмування TypeScript та Node.js, бібліотека React, Express.js, Three.js для 3D-рендерингу, Zustand для управління станом, Prisma ORM та СУБД PostgreSQL для збереження даних.

Результати роботи: створено Full-stack платформу, що дозволяє користувачам досліджувати різні хаотичні аттрактори. Реалізовано математичне ядро з оптимізацією пам'яті, графічний конвеєр із підтримкою до 200 000 точок, систему збереження конфігурацій моделей та режими порівняння.

Наукова новизна та значимість: розроблено та оптимізовано таку архітектуру, що зменшує обчислювальні витрати під час обміну даними та візуалізації. Проєкт дає можливість спостерігати за моделями у браузері із частотою в 60 FPS.

Сфера застосування: може використовуватися у закладах вищої освіти для навчання, а також науковцями для швидкого аналізу динамічних систем.

ЗМІСТ

Скорочення та умовні позначення.....	5
Вступ.....	6
1 Аналіз предметної області та теоретичні основи нелінійної динаміки.....	9
1.1 Поняття нелінійної динаміки та детермінованого хаосу.....	9
1.2 Історичний аспект та становлення теорії хаосу	10
1.3 Фазовий простір та концепція атракторів.....	10
1.4 Фрактальна геометрія та структура дивних атракторів.....	12
1.5 Класифікація систем: безперервні та дискретні.....	12
1.6 Роль інтерактивної візуалізації в дослідженні хаосу	12
2 Математичне та алгоритмічне забезпечення моделювання	14
2.1 Математична формалізація моделей нелінійної динаміки.....	14
2.2 Чисельне інтегрування	15
2.2.1 Порівняння методу Ейлера та методу Рунге-Кутти	15
2.2.2 Алгоритмічна реалізація методу RK4	15
2.3 Проблеми обчислювальної точності та стабільності.....	16
2.4 Алгоритми оптимізації.....	16
2.4.1 Використання типізованих масивів.....	16
2.4.2 Метод циклічного буферу	17
2.5 Перетворення координат та візуальне відображення	17
3 Проєктування архітектури інтерактивного вебсередовища	18
3.1 Аналіз вимог до створення середовища.....	18
3.2 Обґрунтування вибору технологічного стеку.....	19
3.3 Схема розподілу між клієнтом та сервером.....	20
3.4 Логічна структура клієнтської частини.....	20
3.5 Організація управління станом	21
3.6 Проєктування графічного конвеєра та оптимізація пам'яті	22
3.7 Проєктування REST API.....	22
4 Програмна реалізація системи моделювання та візуалізації.....	24
4.1 Програмна реалізація та оптимізація обчислювального двигуна.....	24
4.2 Алгоритм циклу візуалізації.....	25
4.3 Реалізація графічної підсистеми	25
4.4 Сервер програми	26

5 Підсистема управління та механізми взаємодії в середовищі.....	27
5.1 Архітектура підсистеми управління параметрами симуляції.....	27
5.2 Алгоритми візуального порівняння та синхронізації потоків	27
5.2.1 Порівняння систем паралельно.....	27
5.2.2 «Ефект метелика».....	28
5.3 Процес збереження та видалення математичних моделей у бібліотеку.	29
6 Аналіз результатів та оцінка роботи програми	31
6.1 Тестування основних модулів	31
6.2 Оцінка швидкості та оптимізації графіки	32
6.3 Аналіз роботи серверної частини та зберігання даних.....	33
Висновки	34
Джерела	37
Додаток А Визначення понять, пов'язаних з історією теорії хаосу	39
Додаток Б Реалізація обчислень алгоритму RK4 у програмі	41
Додаток В Реалізація двигуна симуляції у програмі	43
Додаток Г Реалізація допоміжного перехоплювача у програмі.....	47
Додаток Ґ Схема даних Prisma.....	49

СКОРОЧЕННЯ ТА УМОВНІ ПОЗНАЧЕННЯ

API — Application Programming Interface (інтерфейс програмування застосунків);

CPU — Central Processing Unit (центральний процесор);

FPS — Frames Per Second (кількість кадрів за секунду);

GPU — Graphics Processing Unit (графічний процесор);

JSON — JavaScript Object Notation (текстовий формат обміну даними);

JSONB — Binary JSON (бінарний формат зберігання JSON в PostgreSQL);

JWT — JSON Web Token (стандарт безпечної передачі інформації);

ORM — Object-Relational Mapping (об'єктно-реляційне відображення);

REST — Representational State Transfer (архітектурний стиль взаємодії компонентів);

RK4 — Runge-Kutta 4th Order (метод Рунге-Кутти четвертого порядку);

WebGL — Web Graphics Library (графічна бібліотека для браузерів).

СУБД — Система управління базами даних.

ВСТУП

Оцінка сучасного стану об'єкта розроблення. Нелінійна динаміка та теорія хаосу на сьогодні відіграють значну роль у багатьох галузях. Найбільше у сучасній науці й пояснює такі складні явища у фізиці, як наприклад механіка, метеорологія, біологія, економіка. Є деякі більш неочевидні — лінгвістика, психологія, хімія та навіть спостерігається у вивченні поведінки деяких тварин [1, с. IX]. Зазвичай моделювання та візуалізація хаотичних систем, таких як дивні атрактори або ітеративні мапи, потребували спеціального програмного забезпечення, які створені спеціально для розв'язання схожих математичних виразів. Наприклад, MATLAB або Wolfram Mathematica.

Проте зараз розвиток таких наукових обчислень спрямований більше на створення вебплатформ. Із появою технологій, як WebGL та JavaScript, з'явилась можливість переносити подібні складні операції в браузер або додаток. Але навіть попри це ще багато існуючих вебінструментів мають недостатню продуктивність для обробки дуже великих масивів даних у реальному часі, тому такий варіант візуалізації подібних систем може бути не найкращим.

Актуальність роботи та підстави для її виконання. Нелінійні динамічні системи критично залежать від початкових умов та параметрів. Навіть найменша зміна параметрів може різко змінити вигляд системи. Через це досліджувати цю галузь стає дуже складно без спеціальних інструментів, що здатні миттєво змінювати вигляд досліджуваної системи.

Актуальність полягає в створенні середовища, в якому можна в реальному часі змінювати параметри та спостерігати, до яких змін в системі це призведе. Такий підхід може допомогти виявити нові властивості атракторів та краще зрозуміти суть хаосу.

Цим середовищем можуть користуватися не лише науковці, а й люди, які зацікавлені в даній сфері.

Мета й завдання роботи. Метою курсової роботи є створення та оптимізація високоефективного інтерактивного вебсередовища для моделювання та візуалізації систем нелінійної динаміки в реальному часі. Для досягнення цієї мети поставлено такі завдання.

- Дослідити та проаналізувати теоретичні основи нелінійної динаміки та відібрати хаотичні системи для подальшого моделювання.
- Дослідити та впровадити оптимізований чисельний інтегратор Рунге-Кутти 4-го порядку.
- Спроекувати та розробити архітектуру візуалізації на базі Three.js та WebGL для відображення траєкторій систем із мінімальною затримкою.
- Реалізувати управління станом системи для можливості зміни параметрів симуляції в реальному часі та підтримки режиму порівняння систем.
- Розробити сервер для автентифікації користувачів та можливості збереження систем із власно підібраними параметрами у бібліотеку — публічно або приватно.
- Експериментально перевірити точність отриманих результатів моделювання та, за необхідності та можливості, оптимізувати та запобігти помилок або небажаної поведінки.

Об'єкт, методи й засоби розроблення. Об'єктом розробки є процес динамічного обчислення та тривимірної візуалізації траєкторій хаотичних систем у режимі реального часу.

Методи дослідження базуються на застосуванні чисельного інтегрування Рунге-Кутти 4-го порядку для розв'язання диференціальних рівнянь та апаратно-прискореній (GPU) візуалізації точкових хмар і ліній.

Засоби для програмної реалізації: React та TypeScript для логіки застосунку, Three.js для 3D-графіки, Zustand для керування глобальним станом інтерфейсу та обчислень. На серверній частині використовувались Node.js та

Express для побудови REST API, Prisma ORM та PostgreSQL для збереження даних.

Можливі сфери застосування. Розроблене середовище може застосовуватися як для наукового, так і для навчального призначення. Науковці можуть застосовувати для швидкої візуалізації фазових портретів нових систем хаосу, в той час як студенти або учні можуть використовувати для навчання.

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ТЕОРЕТИЧНІ ОСНОВИ НЕЛІНІЙНОЇ ДИНАМІКИ

1.1 Поняття нелінійної динаміки та детермінованого хаосу

Нелінійна динаміка — це розділ математики та фізики, що вивчає системи, поведінка яких описується нелінійними рівняннями. Але також можна сказати, що нелінійна динаміка — це майже все, що нас оточує. Зазвичай люди про це не задумуються, але більшість систем в реальному житті є саме нелінійними, в тій чи іншій мірі.

На відміну від лінійних систем, де діє принцип суперпозиції, нелінійні системи демонструють набагато складнішу поведінку. В порівнянні саме нелінійні системи майже неможливо аналітично розв'язати і вони є набагато важчими. Основною причиною цього є те, що лінійні системи можна розбити на частини, окремо розв'язати кожен та скомбінувати їх. Але в реальному житті, хоч і є такі системи (наприклад, рух під дією сили тяжіння без опору), але переважна більшість все ж таки нелінійні. Вони спостерігаються в абсолютно різних сферах життя, і саме тому так важливо їх досліджувати [1 с. 8 – 9, 2].

Також основною характеристикою таких систем є те, що малі зміни в початкових умовах або параметрах можуть призводити до великих та непередбачуваних наслідків у довгостроковій перспективі.

Одним із найбільш вражаючих явищ у нелінійній динаміці є детермінований хаос. Його особливість в тому, що детермінована система без сторонніх збуджень починає поводитися хаотично. Із часом в цій хаотичності почали спостерігати і ряд закономірностей [3].

Хаотичні системи дуже чутливі до навіть найменших змін й з часом ця властивість отримала назву «Ефект метелика». Після проведення експериментів та симуляцій, Едвардом Лоренцом була запропонована передбачуваність: «Чи може помах крил метелика в Бразилії спричинити торнадо в Техасі?» [4]. Від цього і пішла назва цього явища.

1.2 Історичний аспект та становлення теорії хаосу

Народження хаосу відбулося завдяки Анрі Пуанкаре — один із найбільших математиків всіх часів. Під час його дослідження задачі n -тіл він виявив явище чутливості до початкових умов хаосу (додаток А) [5]. Навіть прості гравітаційні взаємодії можуть призводити до надзвичайно заплутаних траєкторій, що неможливо розв'язати аналітично.

Справжній прорив та переродження теорії хаосу відбулися у 1963 році, коли метеоролог Едвард Лоренц за допомогою комп'ютерного моделювання атмосферних потоків виявив перший дивний атрактор. Лоренц помітив, що при незначній зміні одного з дванадцяти параметрів, менш ніж на 0,1%, він отримав кардинально інший кінцевий результат прогнозу погоди.

Сьогодні його модель, система Лоренца, є класичним прикладом системи, що має хаотичну поведінку, що означає, що результат дуже чутливий до зміни початкових параметрів.

1.3 Фазовий простір та концепція атракторів

Для аналізу динамічних систем використовується концепція фазового простору. Це багатовимірний простір змінних динамічної системи, точки якого описують стан цієї системи. Зміни стану системи описується фазовою траєкторією — лінією в фазовому просторі. Оскільки будь-яка точка може слугувати початковою умовою [6], можна зробити висновок, що фазовий простір повністю заповнений фазовими траєкторіями.

Атрактор (від англ. to attract — притягувати) — це точка або безліч точок у фазовому просторі, до яких притягуються всі траєкторії з деякого околу атрактора, що називається також областю тяжіння. Атрактори поділяються на декілька типів (див. рисунок 1.3).

1. Прості аттрактори

Прості атрактори — це стійкі стани (точки рівноваги або замкнені криві) у фазовому просторі, до яких з часом прямують траєкторії динамічної системи. Вони передбачувані та описують стабільні процеси без хаосу.

- Точковий атрактор — це фіксована точка, до якої рухається система. Приклади: падіння книги, затухаючий маятник.
- Граничний цикл — це ізольована та замкнута орбіта. Сусідні траєкторії атрактора не є замкнутими. Вони спіралью рухаються або до граничного циклу, або від нього.
- Торівий атрактор — це система, яка змінює детальні характеристики з часом, але не змінює свою форму. Така система має траєкторію у формі тора [7].

2. Дивні атрактори.

Дивний атрактор — тип атрактору, що визначається певними нелінійними складними системами і характеризується фрактальною структурою. Від простих атракторів відрізняється тим, що дуже чутливий до найменших змін та легко проявляється поведінка хаосу [8].



Рисунок 1.3 — Типи атракторів

1.4 Фрактальна геометрія та структура дивних атракторів

Дивні атрактори не є просто лініями чи поверхнями в класичному розумінні Евклідової геометрії. Вони мають дробову, або фрактальну, розмірність. Це означає, що при будь-якому масштабуванні структура атрактора залишається однаково складною та подібною собі.

Наприклад, атрактор Лоренца має розмірність приблизно 2.06. Це більше ніж поверхневий 2D простір, та менше за об'ємний 3D простір. Фрактальна природа таких систем є ключовою для розуміння того, як нескінченно довга траєкторія може бути обмежена в скінченному об'ємі фазового простору, не перетинаючи саму себе [9]. Така геометрична складність створює складнощі для комп'ютерної візуалізації цих систем, оскільки для нормального відображення структури атрактора необхідно візуалізувати велику кількість обчислювальних точок.

1.5 Класифікація систем: безперервні та дискретні

У нелінійній динаміці виділяють два основних типи математичних моделей, які потребують різних підходів до візуалізації:

1. Безперервні системи. Вони зазвичай описуються системами звичайних диференціальних рівнянь, зміна стану яких відбувається плавно [10 с. 46]. Для їх розв'язування використовуються чисельні методи інтегрування, наприклад, метод Рунге-Кутти. Деякі приклади таких атракторів: атрактор Лоренца, Росслера, Айзави, Чуа, Томаса.
2. Дискретні системи. Зазвичай стан змінюється ітеративно через певні проміжки часу [10 с. 20]:

$$x_{n+1} = f(x_n). \quad (1)$$

Деякі приклади: логістичне відображення, відображення Ікеда, відображення Кліффорда, відображення Хенона

1.6 Роль інтерактивної візуалізації в дослідженні хаосу

Через те, наскільки хаотичні системи непередбачувані та складні, їх графічне представлення та візуалізація — це ледве не єдиний спосіб їх дослідження та вивчення поведінки систем, оскільки аналітично неможливо це зробити. Інтерактивні візуалізаційні середовища можуть допомогти із змінами поведінки в реальному часі при зміні параметрів; із дослідженням областей притягання; із аналізом швидкості розходження траєкторій [11].

Сучасні вебтехнології, такі як WebGL та Three.js, дозволяють реалізувати задачу візуалізації хаотичних систем безпосередньо в браузері. Це забезпечує доступність для представників будь-яких галузей, будь-то наукова сфера чи просто інтерес до нелінійної динаміки та хаосу.

2 МАТЕМАТИЧНЕ ТА АЛГОРИТМІЧНЕ ЗАБЕЗПЕЧЕННЯ МОДЕЛЮВАННЯ

2.1 Математична формалізація моделей нелінійної динаміки

Більшість безперервних хаотичних систем описуються системами звичайних диференціальних рівнянь

$$\frac{du}{dt} = f(u, p),$$

де $u = (x, y, z)^T$ — вектор стану системи;

p — вектор параметрів.

Найпопулярніші базові системи, що демонструють хаотичну динаміку, наведено в табл. 2.1 [12, 13].

Таблиця 2.1 — Найпопулярніші хаотичні базові системи

Назва системи	Математична модель	Класичні параметри хаосу
Система Лоренца	$\begin{cases} dx/dt = \sigma(y - x), \\ dy/dt = x(\rho - z) - y, \\ dz/dt = xy - \beta z. \end{cases}$	$\sigma = 10,$ $\rho = 28,$ $\beta = 8/3.$
Система Рьюсслера	$\begin{cases} dx/dt = -y + z, \\ dy/dt = x + ay, \\ dz/dt = b + z(x - c). \end{cases}$	$a = 0,2,$ $b = 0,2,$ $c = 5,7.$
Атрактор Чуа	$\begin{cases} dx/dt = \alpha(y - x - f(x)), \\ dy/dt = x - y + z, \\ dz/dt = -\beta y, \\ f(x) = m_1 x + 0,5(m_0 - m_1)(x + 1 - x - 1). \end{cases}$	$\alpha = 15,6,$ $\beta = 28,$ $m_0 = -1,143,$ $m_1 = -0,714.$

Система Чена	$\begin{cases} dx/dt = \sigma(y - x), \\ dy/dt = x(s - a) + sy, \\ dz/dt = xy - \beta z. \end{cases}$	$\sigma = 36,$ $\rho = 0,$ $s = 20,$ $\beta = 3.$
-----------------	---	--

Дискретні відображення не потребують чисельного інтегрування, бо вони задаються ітеративною функцією (див. формулу 1). Це спрощує їх математичні обчислення, але водночас і потребує великої швидкості для візуалізації всіх точок відразу.

2.2 Чисельне інтегрування

2.2.1 Порівняння методу Ейлера та методу Рунге-Кутти

Найпростіший метод Ейлера має перший порядок точності, що призводить до можливості дуже легко помилитися. В атраторах краще уникати цієї можливості, оскільки навіть малі зміни або похибки можуть змінити вигляд системи [14, с. 65 – 66]. Саме тому є оптимальним класичний метод Рунге-Кутти четвертого порядку — він більш точний, стабільний та добре підходить для задачі роботи [14, с. 135, 195].

2.2.2 Алгоритмічна реалізація методу RK4

Метод обчислює чотири проміжних коефіцієнти, що дозволяє досягти високої точності:

$$\begin{aligned} k_1 &= f(t_n, u_n); \\ k_2 &= f\left(t_n + \frac{h}{2}, u_n + \frac{h(k_1)}{2}\right); \\ k_3 &= f\left(t_n + \frac{h}{2}, u_n + \frac{h(k_2)}{2}\right); \\ k_4 &= f(t_n + h, u_n + hk_3); \end{aligned}$$

$$u_{n+1} = u_n + h/6(k_1 + 2k_2 + 2k_3 + k_4),$$

де u — вектор стану системи,

$n, n+1$ — індекси, що позначають поточний та наступний крок обчислень,

t — незалежна змінна часу,

h — крок інтегрування. Визначає відстань між сусідніми точками траєкторії,

$f(t, u)$ — векторна функція систем,

$k_i, i = 1, 2, 3, 4$ — проміжні коефіцієнти (нахили), що розраховуються послідовно всередині одного,

Фінальне значення u є зваженим середнім цих чотирьох нахилів.

2.3 Проблеми обчислювальної точності та стабільності.

Під час реалізації двигуна моделювання мовою TypeScript виникло декілька специфічних проблем. По-перше, були проблеми із точністю розрахунків. Використання 64-бітних чисел із плаваючою комою обмежує мінімальний крок інтегрування h [15, с. 161 – 163]. По-друге, відбувається накопичення похибки. Через хаотичну поведінку систем будь-яка навіть найменша похибка округлення буде з часом експоненціально зростати [15, с. 164]. Це призведе до неможливості ідентично відтворити одну й ту саму систему на деяких різних пристроях [15, с. 185–186].

2.4 Алгоритми оптимізації.

Оскільки була задача максимально забезпечити плавність візуалізації (хоча б 60 FPS), без помилок або похибок, то необхідно кожної миті обчислювати тисячі нових точок, при чому запобігаючи можливим помилкам.

2.4.1 Використання типізованих масивів

Класичні масиви TypeScript є об'єктами, що створює велике навантаження на пам'ять та на збирач сміття. Тому в проєкті було використано `Float32Array`, що дозволяє зберігати координати у неперервному бінарному блоці пам'яті.

Використання `Float32Array` забезпечує пряму передачу координат у пам'ять GPU, що максимізує FPS під час візуалізації. Крім того, неперервне розташування даних у пам'яті оптимізує роботу з кешем процесора і усуває ризики замерзання програми.

2.4.2 Метод циклічного буферу

При тривалих та дуже довгих симуляціях пам'ять може переповнюватись через велику кількість точок. Щоб цього уникнути, було реалізовано алгоритм циклічного буферу. Коли кількість точок досягає певного ліміту, нові точки записуються на місце найстаріших. Таким чином немає переповнення пам'яті.

2.5 Перетворення координат та візуальне відображення

Після обчислення координати (x , y , z) вони мають бути перетворені на зображення на екрані користувача. Оскільки різні атрактори мають різні масштаби (Лоренц має масштаб в десятки одиниць, коли Хенон — лише в одиниці), то застосовується алгоритм автоматичного підбору масштабу. Також використовується перспективна камера. Вона дозволяє вільно рухатися в 3D-просторі і в користувача є можливість подивитися на симуляцію з будь-яких сторін та з будь-якої відстані.

3 ПРОЄКТУВАННЯ АРХІТЕКТУРИ ІНТЕРАКТИВНОГО ВЕБСЕРЕДОВИЩА

3.1 Аналіз вимог до створення середовища

Насамперед робота почалась із вивчення та дослідження основної теорії, що знадобиться для розробки даного середовища. Наступним кроком проєктування системи розпочалося із детального аналізу функціоналу, що буде містити в собі проєкт. Було відвідано доволі багато сайтів, щоб подивитись та порівняти, які можливості є в них та які можна реалізувати для цього проєкту, щоб він був якомога більш повним. Головний акцент було зроблено на тому, щоб забезпечити системі характер дослідження. Це означає, що система виступає не просто, як візуалізатор, а й дозволяє її користувачам експериментувати.

Функціональні аспекти. Окрім самої візуалізації системи, багато уваги приділялося механізмам управління параметрами. Кожна модель має свій власний набір параметрів й під час переходу на іншу систему — вони динамічно змінюються. Окрім базових параметрів, що застосовуються для візуалізації моделі, також були додані швидкість та кількість точок (або довжина траєкторії). Швидкість дозволяє спостерігати як в сповільненому, так і в пришвидшеному режимі за симуляцією. В той час довжина траєкторії — кількість намальованих точок на екрані. Візуалізація динамічних систем відбувається нескінченно (якщо того захоче користувач), тому звісно, якщо симуляція буде продовжуватися навіть 5-10 хвилин — кількість точок на екрані буде настільки велика, що може зламати або гальмувати систему. Тому накладаються обмеження, які можна регулювати (максимум — 200 000 точок).

Окрім цього, є такі функціональні можливості, як запустити два різні атрактори поруч один з одним та порівняти їх поведінку при різних параметрах; увімкнути «ефект метелика» — на одному екрані запускаються дві системи одночасно (із мінімальною різницею) і можна побачити, як з часом вони починають по дуже різному себе вести; можливість зберігати та передивлятися системи із власноруч визначеними параметрами.

Технічні обмеження. Критичним з самого початку було і питання плавності візуалізації. Дана система розрахована на приблизно 60 кадрів на секунду, тобто приблизно 16 мс на кадр. Якщо обчислення триватимуть довше, інтерфейс почне помітно зависати, тому всі важкі математичні розрахунки були спроектовані так, щоб не заважати роботі браузера.

3.2 Обґрунтування вибору технологічного стеку

В якості мови програмування було обрано TypeScript із основними фреймворками React, Three.js. Prisma ORM та Express — для роботи з сервером, в якості бази даних було обрано PostgreSQL.

Переваги розробки з React та TypeScript. З використанням TypeScript було легко формалізувати опис математичних систем через інтерфейси. Тобто двигун, що робить всі обчислення, ніколи не отримає некоректні дані, наприклад рядок замість числа. Натомість це могло б призвести до падіння графічного інтерфейсу та серйозних помилок в чисельних методах.

Чому React? Програма складається із багатьох різних частин — 3D-вікно, панель контролю, опис кожної моделі. React дозволяє не тримати це все в одній купі, а відокремлювати. Тобто кожна з частин є модульна і не заважає одна одній. Це також дуже допомагає симуляції проходити більш швидко та плавно.

Графічні можливості Three.js. Three.js було обрано для візуалізації систем. Ця бібліотека дає можливість управляти камерами та світлом, але найважливіше те, що також дозволяє керувати точками вручну. Three.js не розраховані на те, щоб обчислювати тисячі точок щосекундно і зазвичай починає сильно гальмувати. Можливість керувати ними вручну допомогло розробити власну систему, яка миттєво передає координати точок прямо на відео карту.

Сервер на Node.js. Node.js було обрано через його високу швидкість та здатність легко обробляти багато запитів одночасно, оскільки присутня серверна частина в проєкті.

База даних з Prisma ORM. Prisma дозволяє легко взаємодіяти із базою даних (PostgreSQL в цьому випадку), легко пишуться SQL-запити. Також дозволяє працювати з даними, як зі звичайними об'єктами в коді, та автоматично перевіряє їх на помилки, що гарантує правильне збереження інформації.

Керування даними через Zustand. Оскільки координати атракторів змінюються приблизно 60 разів на секунду, то стандартні засоби React просто гальмували б комп'ютер. Zustand дозволяє оновлювати цифри миттєво, завдяки чому інтерфейс працює плавно і не гальмує.

3.3 Схема розподілу між клієнтом та сервером

Майже вся логіка та обчислення відбуваються на пристрої користувача. Через те що клієнт бере на себе більшу майже всю частину роботи, це допомагає усунути затримки, що пов'язані із передаванням даних через інтернет. Кожна обчислена точка розраховується в пам'яті браузера.

Сервер на Node.js відповідає за роль автентифікації користувача та валідації даних. Така схема робить систему стійкою навіть при дуже великій кількості користувачів, оскільки на частині серверу не відбуваються великі дії.

Важливою частиною середовища та архітектури є взаємодія фронтенду із GPU. Математичне ядро CPU спочатку готує дані, а потім передає їх на GPU для візуалізації. Для перегляду схеми див. рисунок 3.1.

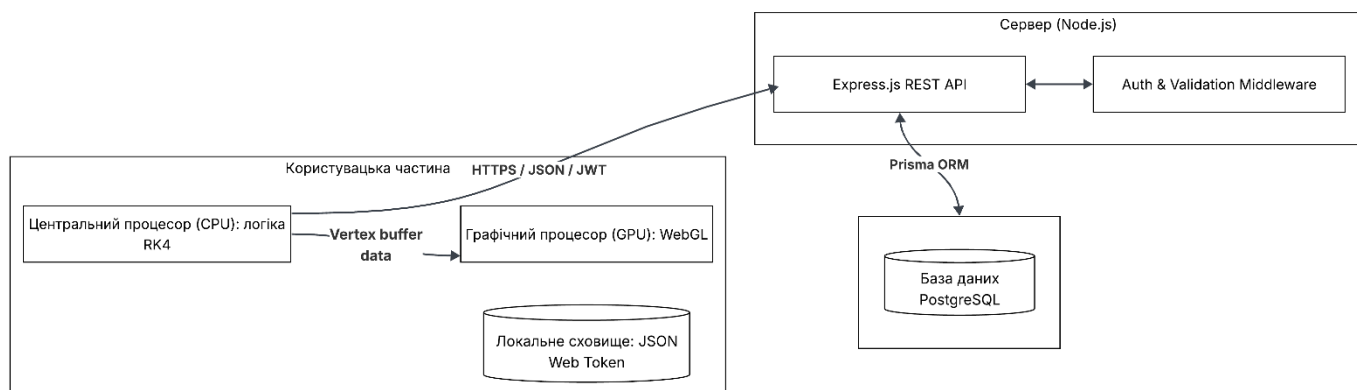


Рисунок 3.1 — Схема розподілу

3.4 Логічна структура клієнтської частини

Логіка клієнтської частини побудована на слабких зв'язках (loose coupling). Тобто математичний двигун не знає, які дані будуть відображені, а візуалізатор не знає, яким чином будуть розраховані координати.

Реєстр систем. Цей модуль містить математичні визначення та опис всіх атракторів. В кожному атракторі всередині окремо є функція для обчислення похідних. Така архітектура забезпечує легке додавання нових систем і не потребує переписання логіки обчислень або візуалізації.

Двигун симуляції. Ця частина відповідає за чисельне інтегрування, тобто окремо робить розрахунки від графічної частини. Відповідає за реалізацію оптимізованого алгоритму RK4, контроль кроку часу та послідовне оновлення координат стану.

Модуль візуалізації. Цей модуль відповідає виключно за візуалізацію — трансформує розраховані (в попередньому модулі) координати у WebGL-атрибути та передає їх в геометрію відео карти. І оскільки в програмі реалізовані як безперервні, так і дискретні системи, то ще визначає як саме відображати симуляцію.

Контролер інтерфейсу. Набір React-компонентів для управління параметрами та візуалізації за допомоги повзунків та кнопок. Синхронізує дії користувача із глобальним сховищем, що робить зміни моментальними. Для перегляду діаграми див. рисунок 3.2.

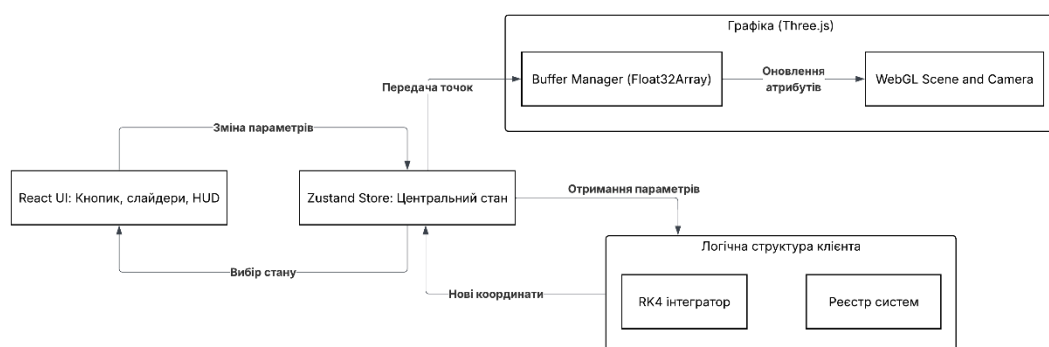


Рисунок 3.2 — Діаграма логічних компонентів

3.5 Організація управління станом

Як вже раніше було сказано, для забезпечення миттєвого управління станами було використано бібліотеку Zustand.

Транзитні оновлення (Transient Updates). Чому не були обрані бібліотеки, як наприклад Redux чи Context API для зберігання? Їх основна проблема в тому, що вони змушують при кожній зміні оновлювати всі дані. В цій програмі координати точок змінюються зі швидкістю 60 разів на секунду, тому можна уявити, як перенавантажить систему оновлення всіх елементів за такий короткий проміжок часу. Тому був використаний підхід транзитних оновлень — Simulation Loop (міст між React-ом та циклом анімації) напряду спілкується із Store [16].

Управління глобальними параметрами. Zustand також зберігає ряд параметрів для моделі. Наприклад, коли користувач завантажує чиюсь модель із бібліотеки, бібліотека просто перезаписує стан Store.

3.6 Проєктування графічного конвеєра та оптимізація пам'яті

Використання BufferGeometry. Замість створення тисяч окремих об'єктів створюється лише один об'єкт геометрії. Це дозволяє візуалізувати десятки тисяч точок за один виклик і сильно знижує навантаження на процесор [17].

Типізовані масиви. В програмі було обрано використовувати Float32Array. Це дозволяє передавати координати з CPU на GPU майже миттєво. Це також гарантує мінімальну затримку між розрахунком точки та її появою на екрані.

Оптимізація пам'яті. Завчасно виділена пам'ять при ініціалізації зупиняє програму від створення нових об'єктів під час роботи обчислень. Такий підхід запобігає паузам на збирання сміття і забезпечує стабільну частоту кадрів.

3.7 Проєктування REST API

Взаємодія між клієнтською частиною та сервером реалізована на принципах архітектури REST. Було спроектовано набір API-ендпоінтів для забезпечення реєстрації користувачів, логіну та управління моделями. Основний формат обміну даними — JSON. Для захисту приватних даних було обрано реалізувати автентифікацію через JWT (JSON Web Tokens) [18]. Це дозволяє передавати ідентифікатор користувача у заголовок кожного запиту, не перевантажуючи базу даних сесіями. Див. таблицю 3.1.

Таблиця 3.1 — основні API Endpoints системи.

API-ендпоінт	Опис
POST /api/auth/register	Створює новий профіль користувача
POST /api/auth/login	Автентифікація та видача JWT-токена
GET /api/users/:userId	Отримання даних профілю користувача
POST /api/auth/forgot-password	Запит на відновлення пароля
POST /api/auth/reset-password	Встановлення нового пароля за допомогою токена
POST /api/auth/change-password	Зміна пароля зсередини особистого кабінету
GET /api/presets	Отримання списку всіх моделей
POST /api/presets	Збереження поточної моделі
DELETE /api/presets/:id	Видалення моделі з бібліотеки
GET /users/:userId/presets	Отримання списку всіх моделей користувача

4. ПРОГРАМНА РЕАЛІЗАЦІЯ СИСТЕМИ МОДЕЛЮВАННЯ ТА ВІЗУАЛІЗАЦІЇ

4.1 Програмна реалізація та оптимізація обчислювального двигуна

Одним із найважливіших компонент програми є клас `SimulationEngine`. Він відокремлений від інших частин програми та займається лише математичними обчисленнями.

Реалізація чисельного інтегратора. В основі обчислень лежить метод Рунге-Кутти 4-го порядку (див. розділ 2.2.2). Програмна реалізація оптимізована та використовує підхід з управлінням пам'яттю: замість ініціалізації нових масивів на кожному кроці розрахунку коефіцієнтів, заздалегідь виділяється блок пам'яті `scratch` типу `RK4Scratch`, яким і керує двигун. Алгоритм розрахунку проміжних коефіцієнтів RK4 у програмі наведено у Додатку Б.

Програмний інтерфейс об'єкта `scratch`:

```
interface RK4Scratch {
    y2: Float32Array; // буфер для k2
    y3: Float32Array; // буфер для k3
    y4: Float32Array; // буфер для k4 }.
```

Це дозволяє мінімізувати вплив збирача сміття. В інакшому випадку, перевантаження на збирач сміття може призвести до різкого падіння продуктивності або просто зламати програму.

Механізми стабільності та перевірки обчислень. У головному методі класу реалізований контроль за ходом симуляції (функція `step()`). Оскільки хаотичні системи часто можуть «ламатися», двигун викликає модуль `SimulationValidator` на кожному кроці для запобігання подібним помилкам візуалізації. У випадку, якщо координати наступного кроку симуляції недійсні — відразу зупиняє симуляцію. Більш детальний алгоритм роботи цього класу наведено у додатку В.

4.2 Алгоритм циклу візуалізації

Взаємодія між математичним ядром та інтерфейсом React реалізована через власно створений гук (перехоплювач) `useSimulationLoop`. Кожен кадр програма виконує алгоритм:

1. Зчитування поточних параметрів із Zustand Store. Дозволяє отримувати доступ до даних миттєво та без затримки.
2. Виклик методу `engine.step()` (див. розділ 4.1), який на основі дельти часу та швидкості розраховує масив нових точок.
3. Синхронізація та оновлення глобального стану через `addPoints`.

Детальніше про цей алгоритм наведено у додатку Г.

В програмі виконуються саме такі розрахунки масиву нових точок задля забезпечення візуальної плавності. Програма може запускатися з різних пристроїв, що мають різну роздільну здатність або продуктивність. Тому програма намагається оптимізувати свою роботу під кожен пристрій. Якщо комп'ютер працює повільно, то двигун автоматично збільшує кількість кроків інтегрування за один кадр:

```
const stepsPerFrame = Math.min(500, Math.max(1,
Math.floor((delta * speed) / dt)));
```

Дельта — це час, за який пристрій обробляє один кадр, а швидкість встановлена користувачем. Це забезпечує, що атрактор буде будуватися з однаковою швидкістю на різних пристроях та моніторах.

4.3 Реалізація графічної підсистеми

Для відображення траєкторій у 3D-просторі використано компонент `SimulationVisualizer`, побудований на базі `Three.js`. Дана програма працює із трьох вимірними (та двовимірним) дивними атракторами та із дискретними відображеннями. Для кожного виду є власний клас.

Основною задачею та складністю було відображення великої кількості (понад 100 000 – 150 000) точок без падіння продуктивності. Було вирішено, що програма буде працювати із атрибутами THREE.BufferGeometry [19]. Дані про позиції точок зберігаються у типізованому масиві Float32Array.

Для економії ресурсів використовується властивість updateRanges. Замість того, щоб кожного кадру відправляти на відео карту весь масив координат, відправляється лише мала частина, в якій з'явилися нові точки:

```
posAttr.updateRanges.push({
    start: startIdx * 3,
    count: (currentCount - startIdx) * 3});
```

4.4 Сервер програми

Сервер програми реалізовано на базі Node.js та Express.js, що забезпечує швидку обробку запитів до бази даних PostgreSQL через Prisma ORM [20].

Різні аттрактори містять різну кількість та назви параметрів, тому звичайна реляційна модель з фіксованими стовпцями була б незручною та неправильною. Тому всі математичні параметри зберігаються у файлі типу JSONB у PostgreSQL. Наприклад, параметри аттрактора Лоренца виглядатимуть в базі даних ось так: «{"sigma": 10, "rho": 28, "beta": 2,66}». Аналогічно з усіма іншими аттракторами. Схему даних Prisma наведена у Додатку Г.

Це дозволяє системі бути розширюваною. Можна додавати нові математичні моделі із абсолютно іншими параметрами і база даних при цьому не зламається.

Для захисту персональних бібліотек та доступу до функцій збереження використовується JSON Web Tokens. При кожному запиті до API сервер перевіряє заголовок авторизації. Це гарантує приватність користувача.

5 ПІДСИСТЕМА УПРАВЛІННЯ ТА МЕХАНІЗМИ ВЗАЄМОДІЇ В СЕРЕДОВИЩІ

5.1 Архітектура підсистеми управління параметрами симуляції

Підсистема виступає в якості контролера, що забезпечує зв'язок між діями користувача із двигуном обчислень. Кожна зміна значення в інтерфейсі запускає подію, яка оновлює сховище Store. Бібліотека Zustand дозволяє математичному ядру отримувати нові значення коефіцієнтів в реальному часі без переривання симуляції або обчислень. Це забезпечує неперервність симуляції [16].

Щоб зрозуміти перевагу такого підходу, буде доцільно порівняти із іншими рішенням. Наприклад, якщо б замість Zustand взяти React State або Context API. Кожна зміна параметра, кожен рух повзунка параметра користувачем викликали б повне перемальовування симуляції. React намагався б щоразу заново створити 3D-об'єкт та всі інші елементи інтерфейсу через високу кількість кадрів на секунду. Це призвело б до сильного падіння FPS та сильних зависань графіки.

5.2 Алгоритми візуального порівняння та синхронізації потоків

Одна із задач для створення цієї програми було дати можливість користувачеві порівнювати різні моделі, із різними параметрами між собою та спостерігати за «ефектом метелика».

5.2.1 Порівняння систем паралельно

Програма створює дві незалежні симуляції — ліву та праву. Кожна з них має власну копію SimulationEngine та свій масив точок. Логіка синхронізації забезпечує однаковий крок часу для обох систем, що дозволяє проводити порівняння динаміки різних атракторів.

Є можливість управляти обома симуляціями як разом, так і окремо. Передбачені кнопки паузи та перезавантаження для обох систем, так само як і

кнопку синхронізації камери, щоб одночасно управляти камерою в обох просторах. Вигляд у програмі показано на рисунку 5.1.

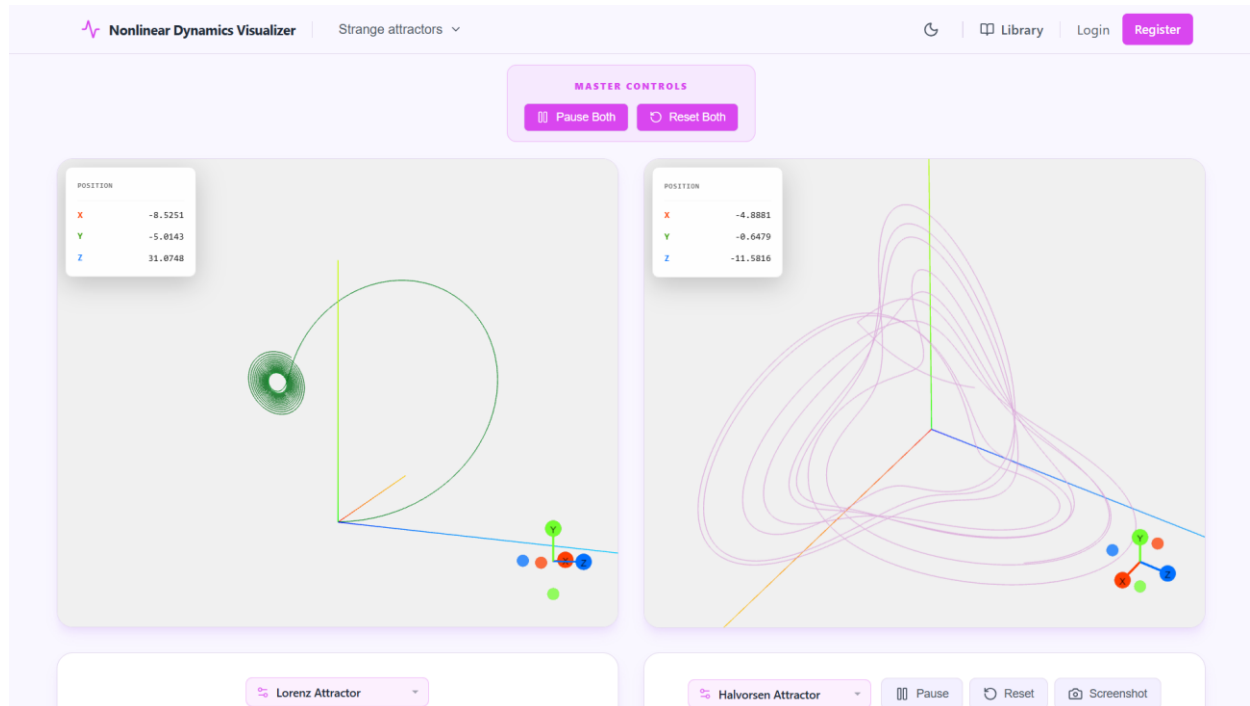


Рисунок 5.1 — Приклад вигляду функції порівняння у середовищі

5.2.2 «Ефект метелика»

У цьому режимі програма запускає дві ідентичні системи, але вводить мікроскопічне відхилення, яке можна налаштувати від 0,00001 до 0,01, у початковий вектор стану однієї з них. Вигляд у програмі показано на рисунку 5.2.

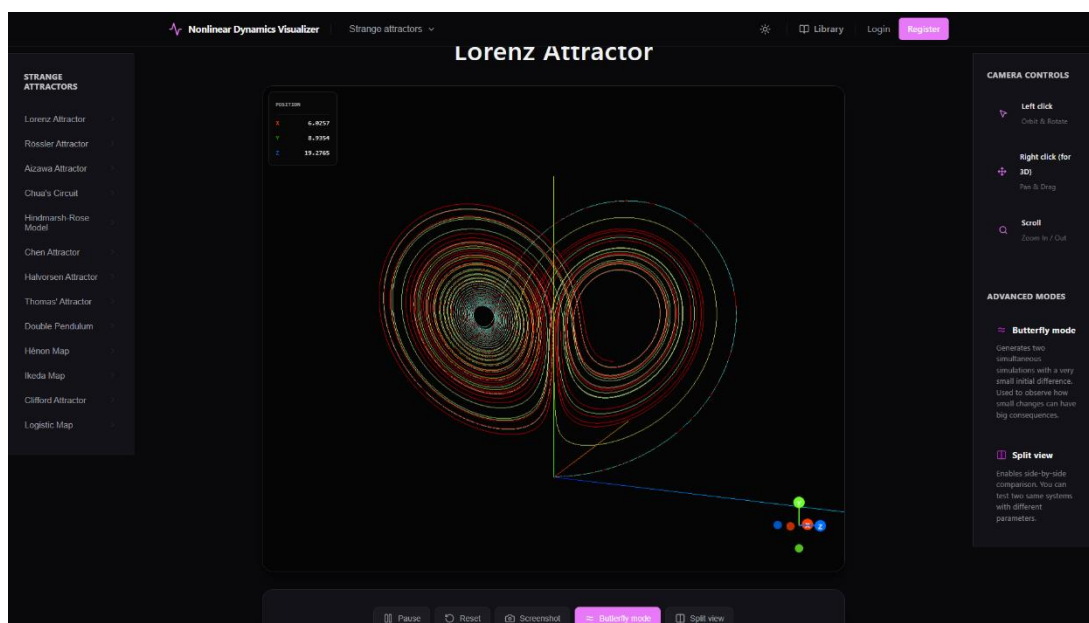


Рисунок 5.2 — Приклад вигляду ефекту метелика у середовищі

5.3 Процес збереження та видалення математичних моделей у бібліотеку

В середовищі передбачена можливість зберігати симуляцію із власноруч встановленими параметрами або кольоровою палітрою — в приватну або публічну бібліотеку.

Коли користувач зберігає модель, модуль PresetActions зберігає параметри поточного стану Store, включаючи тип системи, значення всіх повзунків та налаштування візуалізатора. Ці дані перетворюються в JSON-об'єкт. При завантаженні моделі відбувається зворотний процес: програма форматує отриманий JSON та примусово оновлює Store. Оновлення Store автоматично переналаштовує SimulationEngine та сприяє миттєвому початку нової симуляції з налаштованими параметрами [21].

Приклад вигляду спільної бібліотеки та бібліотеки користувача на рисунку 5.3 та 5.4.

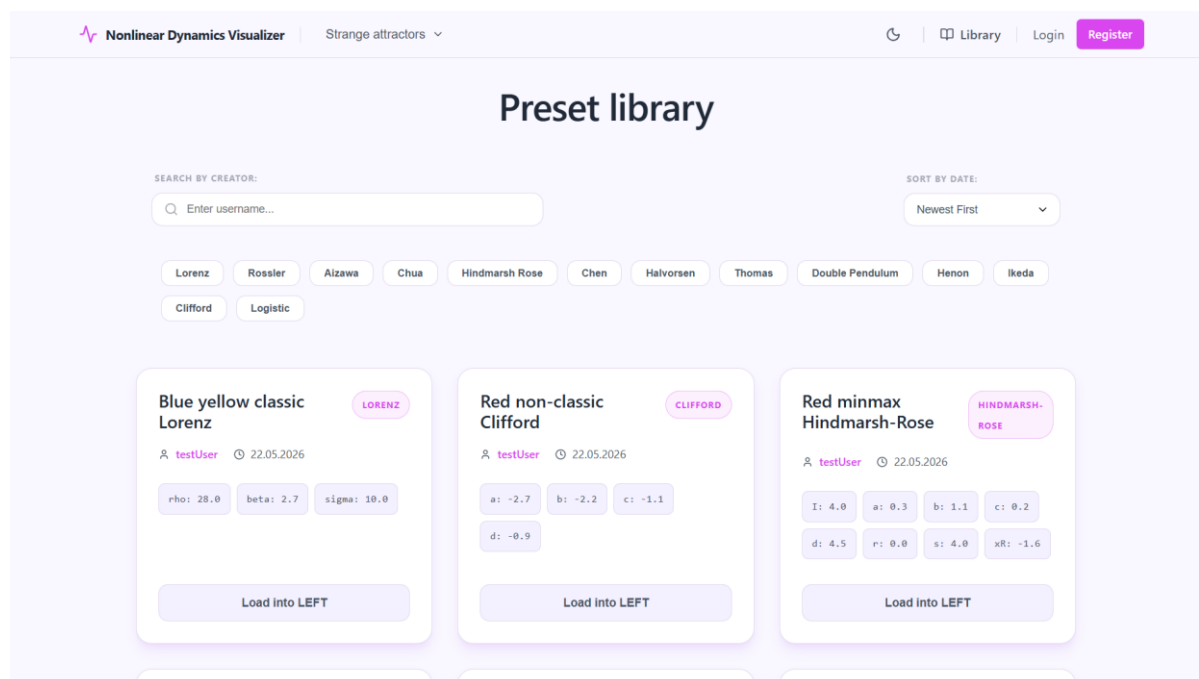


Рисунок 5.3 — Приклад спільної бібліотеки

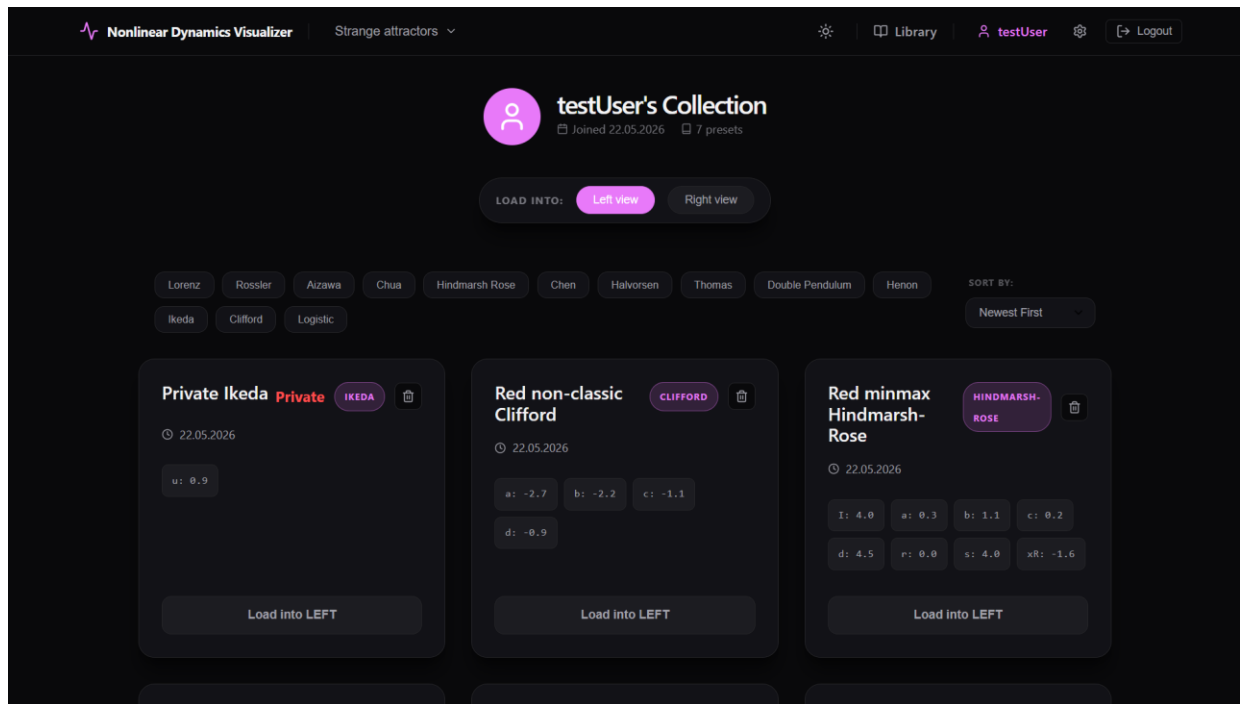


Рисунок 5.4 — Приклад особистої бібліотеки

6 АНАЛІЗ РЕЗУЛЬТАТІВ ТА ОЦІНКА РОБОТИ ПРОГРАМИ

6.1 Тестування основних модулів

Після завершення програмної розробки, було проведено повне тестування програми — в першу чергу власноруч перевірялись всі основні функції, як поводитись себе симуляції при різних параметрах, наскільки швидко працює зміна параметрів, перевірка всього додаткового функціоналу (наприклад, порівняння). Також, оскільки в програмі є сервер, перевірялись виклики API та їх доступність — всі моделі зберігаються коректно, якщо користувач зареєстрований; модель не може видалити ніхто, окрім користувача, що її створив; приватні моделі не бачить ніхто, окрім цього користувача.

Також були проведені декілька тестів у програмі. Всі тести в програмі проводились з допомогою Vitest. Тестувати абсолютно всі файли підряд було б не дуже доцільно, оскільки їх забагато та багато з них не дадуть суттєвої інформації, тому перед початком написання тестів було проведено аналіз, що протестувати необхідно, а що можна пропустити.

Був протестований метод чисельної інтеграції RK4. Була дуже важлива саме правильна робота цього методу, оскільки це є основою симуляцій. Перевірялось декілька тривіальних тестів та проводилося моделювання гармонічного осцилятора та лінійного росту. Наприклад, при розв'язанні рівняння, при якому похідна від x за часом t дорівнює x й з початковою умовою $x=1$, система порівнювала чисельний результат програми з аналітичним значенням функції. Програма показувала, що немає великої похибки і точність доволі висока. Це є важливим, оскільки, як вже раніше в роботі була сказано, навіть невелика похибка може спотворити симуляцію і вивести неправильний результат.

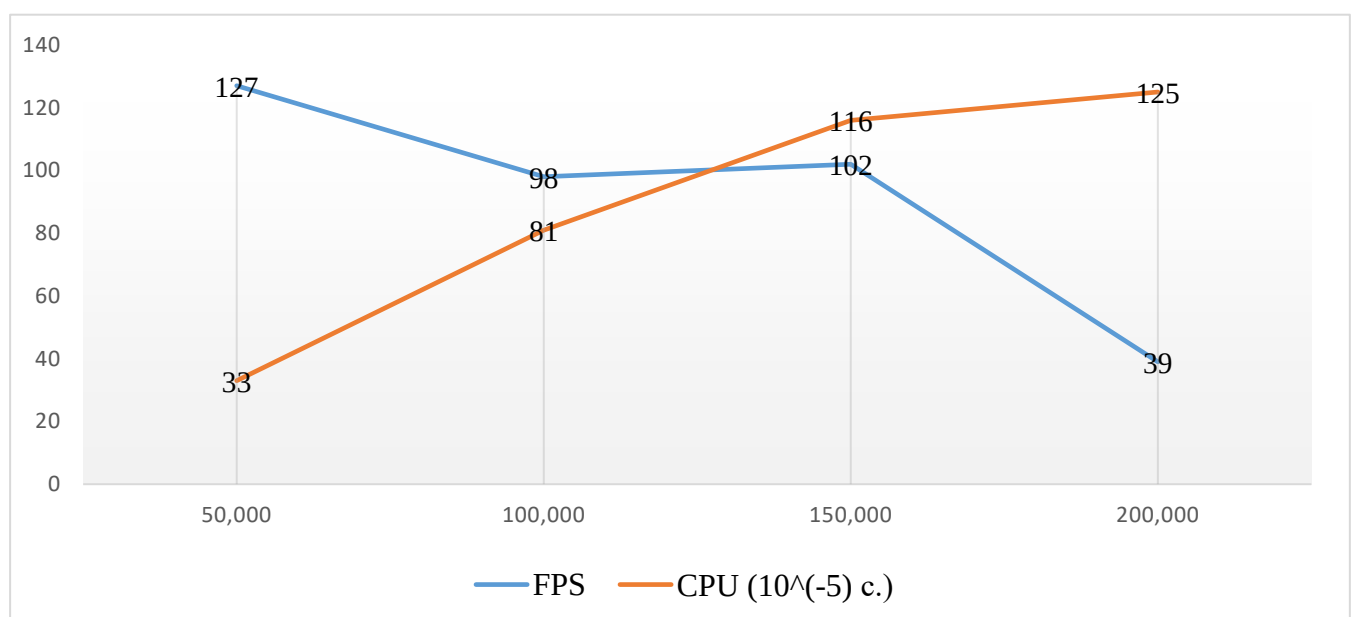
Також були тести для, наприклад, перевірки реєстру систем та роботи всіх візуальних частин інтерфейсу (всі кнопки та повзунки). Всі тестування в програмі пройшли успішно.

6.2 Оцінка швидкості та оптимізації графіки

Однією з головних задач для цієї програми було забезпечити користувачу плавність роботи симуляції та високу продуктивність. Тому важливим етапом було проаналізувати продуктивність графіки.

Дослідження частоти кадрів та навантаження на систему. Варіант із використанням `BufferGeometry` та типізованими масивами `Float32Array` дозволяють продуктивності бути стабільною. Для тестування перевірки FPS та CPU були додані спеціальні функції в програму, що дозволяли бачити в реальному часі, як саме змінюються ці параметри. При налаштуваннях повзунків довжина = 200 000 точок та швидкість = 9, програма залишалась більш менш стабільною весь час до моменту настання 200 000 точок.

FPS протягом всієї симуляції залишалось приблизно в межах 100 – 120 (хоч інколи могло бути і трохи нижче, але не менше ніж 65). При настанні максимуму точок на екрані, 200 000, програма почала працювати не так продуктивно через більше навантаження. Спостерігалось, що в середньому FPS було в межах від 35 до 80. На графіку 6 наведені результати тестування. Робота CPU в середньому збільшилась в 2,5 – 4 рази від початку симуляції (0 точок) до її завершення (200 000 точок).



Графік 6 — Тестування продуктивності

6.3 Аналіз роботи серверної частини та зберігання даних

Було проаналізовано серверну частину, її взаємодію із клієнтською частиною та стабільність взаємодії із базою даних.

Найголовніше в частині серверу — це забезпечити безпеку користувачу. В програмі є можливість не лише створювати публічні моделі, а й приватні, для особистого використання, та можливість їх видаляти. Перевірки були проведені не лише із браузером, але й з інструментом Postman, з яким дуже зручно перевіряти API. Користувач, що не зайшов до свого акаунту, або користувач, що зайшов, але не є тим хто створив конкретну модель, не можуть видалити її або передивитися приватні конфігурації. При спробі видалення система покаже помилку: «Доступ заборонено. Токен автентифікації не надано».

Також були перевірені логін та реєстрація акаунту, відновлення паролю та його зміна. Будь-який користувач не зможе зайти в будь-який акаунт — проводяться перевірки на співпадіння паролю (що хешується і зберігається в СУБД відповідним чином) та електронної пошти акаунту. Без знання паролю на акаунт зайти або змінити його неможливо. А якщо користувач і забув пароль — то посилання на його відновлення надсилається йому на пошту (хоча в цій програмі це реалізовано локально і як симуляція).

ВИСНОВКИ

У процесі виконання курсової роботи було проведено теоретичне дослідження за темою «Нелінійна динаміка та теорія хаосу», були проведені аналізи, які можливості додати у програму та як оптимізувати роботу симуляції, які краще застосовувати інструменти, та було реалізовано інтерактивне вебсередовище.

Оцінка одержаних результатів та їх відповідність сучасному рівню технологій. Головною задачею і результатом роботи було створення зручного та продуктивного інтерфейсу для перегляду та вивчення дивних атракторів. Використання методу Рунге-Кутти 4-го порядку дозволило отримати велику точність результатів і не давати програмі видавати неправильну симуляцію — був проведений тест, що довів велику точність розрахунків і похибку обчислень не вище 10^{-6} . Це цілком відповідає затребуваним вимогам.

Для графічної продуктивності програма використовує WebGL-рендеринг. Завдяки використанню BufferGeometry та Float32Array програма працює плавно та працює стабільно із частотою 60 кадрів на секунду навіть при відображенні більшої кількості точок траєкторії. Це дозволяє візуалізації в браузері працювати на рівні із такими застосунками, як MATLAB та Wolfram Mathematica.

Програмна реалізація та архітектурні рішення. Створена програма базується на Full-stack архітектурі — присутній як клієнтська частина, так і серверна.

Клієнтська частина, реалізована на React та TypeScript, використовує систему управління станом Zustand. Це дозволило використати механізм «транзитних оновлень», за якого не перенавантажувється програма і обходить рендеринг React компонентів.

Серверна частина на Node.js, Express.js та Prisma ORM реалізує захищене збереження результатів досліджень. Модель бази даних PostgreSQL гнучка завдяки використанню типу JSONB для зберігання параметрів систем, що

забезпечує користувачеві можливість додавати власні математичні моделі без додаткових зусиль.

Також в програмі була розроблена автентифікація через JWT-токени та процеси відновлення і зміни паролів, що забезпечує користувачу безпеку.

Наукова та технічна значущість роботи. Наукова значущість проєкту полягає у розробці середовища, яке дозволяє не просто спостерігати за хаотичними системами без можливості ними керувати, а активно досліджувати їх властивості, керуючи ними самостійно. Реалізація режиму «ефекту метелика» та механізму паралельного порівняння двох систем надає користувачу більш наглядно спостерігати за системами та робити власні висновки.

Технічна значущість спостерігається у створенні оптимізованого графічного конвеєра з механізмом `updateRanges`. Це мінімізує навантаження при копіюванні даних із CPU в GPU, що є важливим для швидкості роботи веббраузерів та збереження енергії пристрою.

Ступінь впровадження та практична цінність. Розроблене середовище має потенціал бути впровадженим у декількох сферах:

1. Вища освіта. Програма може слугувати середовищем для студентів фізико-математичних та комп'ютерних спеціальностей при вивченні курсів, що безпосередньо пов'язані із нелінійною динамікою та теорією хаосу.
2. Наукові дослідження. Платформа дає можливість швидко створювати й тестувати моделі нових систем, аналізувати симуляцію та їх поведінку під час зміни параметрів.

Також завдяки присутності бібліотеки є можливість обмінюватися із іншими користувачами власними цікавими конфігураціями.

Перспективи подальших досліджень та розвитку проєкту. Робота має вже базу для подальшого розширення функціоналу. Можливо не лише розширити кількість систем, а й покращити програму, для обчислення не лише

сотень тисяч точок, а й мільйонів одночасно. Також перспективним є можливість додати парсер математичних виразів. Це дозволить користувачеві вводити власні диференціальні рівняння в інтерфейсі без потреби додавати їх власноруч до реєстру систем. Можна також створити експорт траєкторій у відео формати або 3D-моделі.

ДЖЕРЕЛА

1. Steven H. Strogatz. Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering [second edition], 2015. IX с.
2. Phil Dyke, Roger Whitworth. Guide to Mechanics, 2001. 319 с.
3. Research group NTUU “Igor Sikorsky Kyiv Polytechnic Institute”. DETERMINISCTIC CHAOS. URL: <https://chaos.kpi.ua/en> (дата звернення: 05.05.2026).
4. Edward N. Lorenz. Predictability: Does the Flap of a Butterfly's Wings in Brazil Set Off a Tornado in Texas? 1972.
5. Christian Oestreicher. A history of chaos theory, 2007. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC3202497> (дата звернення: 05.05.2026).
6. А. М. Федорченко. Теоретична механіка. Київ: Вища школа: 1975. 516 с.
7. Types of Attractors. URL: https://www.uobabylon.edu.iq/eprints/paper_1_24607_154.pdf (дата звернення: 05.05.2026).
8. Schuster, H. G. Deterministic Chaos, 1989. 105 – 106 с.
9. Mandelbrot, B. The Fractal Geometry of Nature, 1982. 193 – 198 с.
10. Sprott J. C. Chaos and Time-Series Analysis, 2003.
11. Parker T. S., Chua L. O. Practical Numerical Algorithms for Chaotic Systems, 2012. 69 – 128 с.
12. Julien C. Sprott. Strange Attractors: Creating Patterns in Chaos, 1993. 326 – 327 с.
13. Petra Augustova and Zdenek Beran. Characteristics of the Chen Attractor, 2013. 306 с.
14. Butcher, J. C. Numerical Methods for Ordinary Differential Equations (2nd ed.), 2008.
15. Goldberg D. What Every Computer Scientist Should Know About Floating-Point Arithmetic, 1991. 161 – 164, 185 – 186 с.

- 16.Zustand: open-source software repository. GitHub. URL:
<https://github.com/pmndrs/zustand> (дата звернення: 20.05.2026).
- 17.B. Maccartney. Real-Time 3D Graphics with WebGL 2, 2018. 112 – 115 с.
- 18.Subramanian V. Pro MERN Stack. 2nd ed., 2019. 375 – 378 с.
- 19.Three.js Documentation: BufferGeometry. URL:
<https://threejs.org/docs/#api/en/core/BufferGeometry> (дата звернення:
20.05.2026).
- 20.Prisma Data Guide. URL: <https://www.prisma.io/docs/orm/prisma-client>
(дата звернення: 20.05.2026).
- 21.ECMA International. ECMA-404: The JSON Data Interchange Standard.
2nd ed. 2017.

ДОДАТОК А

Визначення понять, пов'язаних з історією теорії хаосу

- Принцип причинності. Кожен наслідок має попередню безпосередню причину.
- Детермінізм. Філософське твердження, що кожна подія фізично визначається нерозривним ланцюгом попередніх подій.
- Передбачуваність. Це стосується ступеня, до якого можна зробити правильний прогноз стану системи як якісно, так і кількісно.
- Модель. Шаблон, план, представлення або опис, призначений для демонстрації структури або роботи об'єкта, системи або концепції.
- Динамічна система. Система, яка змінюється з часом як причинно-наслідковим, так і детермінованим чином; тобто її майбутнє залежить лише від явищ з її минулого та сьогодення (причинність), і кожна задана початкова умова призведе лише до одного заданого пізнішого стану системи (детермінізм). Системи, які є шумними або стохастичними, у сенсі демонстрації випадковості, не є динамічними системами, і теорія ймовірностей є такою, яка застосовується до їх аналізу.
- Фазовий простір. Абстрактний простір, у якому представлені всі можливі стани системи, причому кожен можливий стан системи відповідає одній унікальній точці у фазовому просторі.
- Чутливість до початкових умов. Це коли зміна однієї змінної має наслідком експоненціальну зміну в системі.
- Інтегрована система. У математиці це стосується системи диференціальних рівнянь, для якої можна знайти розв'язки. У механіці це стосується квазіперіодичної системи.

- Лінійна система. Система називається лінійною, коли ціле точно дорівнює сумі її компонентів. Атрактор — множина, до якої динамічна система еволюціонує після достатньо тривалого часу.

- Характерний час Ляпінова. Характерний час системи визначається як затримка, коли зміни від початкової точки множаться на 10 у фазовому просторі.

- Зворотний зв'язок. Реакція на інформацію, яка або збільшує ефекти (позитивний зворотний зв'язок), або зменшує їх (негативний зворотний зв'язок), або викликає циклічне явище.

- Самоподібність. Це означає, що об'єкт складається з субодиноць та підсубодиноць на кількох рівнях, які (статистично) нагадують структуру цілого об'єкта. Однак у повсякденному житті обов'язково існують нижні та верхні межі, за якими застосовується така самоподібна поведінка.

- Фрактал. Це геометричний об'єкт, що задовольняє два критерії: самоподібність та дробова розмірність.

- Фрактальна розмірність. Нехай об'єкт у n -вимірному просторі покритий найменшою кількістю відкритих сфер радіуса r . Фрактальна розмірність дорівнює $\log(N)/\log(1/r)$, коли r прагне до 0.

ДОДАТОК Б

Реалізація обчислень алгоритму RK4 у програмі

```

export interface RK4Scratch {

  y2: Float32Array;

  y3: Float32Array;

  y4: Float32Array;}

export function createRK4Scratch(dimension: number): RK4Scratch {

  return { y2: new Float32Array(dimension),

    y3: new Float32Array(dimension),

    y4: new Float32Array(dimension),}; }

export function rk4(

  out: StateVector, y: StateVector, t: number, dt: number,

  derivative: DerivativeFn, scratch: RK4Scratch,): void {

  const n = y.length;

  const { y2, y3, y4 } = scratch;

  const dtHalf = dt * 0.5;

  const dtSix = dt / 6.0;

  // Evaluate k1 and project half-step to y2

  const k1 = derivative(y, t);

  for (let i = 0; i < n; i++) {

    y2[i] = y[i] + k1[i] * dtHalf;}

  // Evaluate k2 and project half-step to y3

  const k2 = derivative(y2, t + dtHalf);

  for (let i = 0; i < n; i++) {

    y3[i] = y[i] + k2[i] * dtHalf;}

  // Evaluate k3 and project full-step to y4

  const k3 = derivative(y3, t + dtHalf);

```

```
for (let i = 0; i < n; i++) {  
    y4[i] = y[i] + k3[i] * dt;}  
  
// Evaluate k4 and combine average into buffer 'out'  
const k4 = derivative(y4, t + dt);  
  
for (let i = 0; i < n; i++) {  
    out[i] = y[i] + dtSix * (k1[i] + 2.0 * k2[i] + 2.0 * k3[i] + k4[i]);}}
```

ДОДАТОК В

Реалізація двигуна симуляції у програмі

```
export class SimulationEngine {
  private lastSavedPoint: StateVector | null = null;
  private scratch: RK4Scratch;

  /** Creates a new simulation engine instance. */
  constructor(dimension: number = 3) {
    this.scratch = createRK4Scratch (dimension);}

  /**
   * Advances the simulation forward in time (this function is called every
   frame).
   *
   * Runs safety checks to stop glitches or math explosions before updating
   the position.
   *
   * @param lastPoint - The current position of an object
   * @param delta - Time since last frame
   * @param speed - Simulation speed (is chosen by user)
   * @param derivative - Function that tells where to go next
   * @returns An array of coordinates that object passed during this step
   time
   *
   * (in simpler words - it is the piece of trajectory that has
   been drawn).
   */
  public step(
    lastPoint: StateVector,
    delta: number,
    speed: number,
    derivative: (state: StateVector, t: number) => StateVector):
    StateVector[] {
    if (!this.lastSavedPoint) {
      this.lastSavedPoint = lastPoint;}

    const dt = 0.002; // fixed step - 0.002s (means that one step takes
2ms)

    // Cap maximum steps per frame to prevent the app from freezing
    const stepsPerFrame = Math.min(
      500,
```

```

    Math.max(1, Math.floor((delta * speed) / dt)),);
const frameBatch: StateVector[] = [];
let currentPoint = lastPoint;
for (let i = 0; i < stepsPerFrame; i++) {
    const nextPoint = new Float32Array(currentPoint.length);
    rk4(nextPoint, currentPoint, 0, dt, derivative, this.scratch);
    // Safet checks
    if (!SimulationValidator.isValidPoint(nextPoint as any)) {
        AppLogger.warn(
            "Simulation Engine: NaN or Infinite point detected. Stopping
integration.",
            { lastValid: currentPoint,
              invalid: nextPoint,,});
        break;}
    if (
        !SimulationValidator.isStableStep(currentPoint, nextPoint as any,
10000)) {
        AppLogger.warn(
            "Simulation Engine: Mathematical explosion (divergence)
detected. Stopping integration.",
            {from: currentPoint,
              to: nextPoint,,});
        break; // Stop updates instantly if the physics engine breaks}
    currentPoint = nextPoint as any;
    // To avoid saving too much points - save a point only if it moved
far enough; ignore if it (almost) stays in place.
    const dx = currentPoint[0] - this.lastSavedPoint[0];
    const dy = currentPoint[1] - this.lastSavedPoint[1];
    const dz = currentPoint[2] - this.lastSavedPoint[2];
    const distSq = dx * dx + dy * dy + dz * dz;
    if (distSq >= 0.005) {
        const savedPt = Array.from(currentPoint);
        frameBatch.push(savedPt);
        this.lastSavedPoint = savedPt;}}

```

```

    // If moving very slowly, save at least one point so the line doesn't
    disappear

    if (
        frameBatch.length === 0 &&
        stepsPerFrame > 0 &&
        currentPoint !== lastPoint) {
        const fallbackPt = Array.from(currentPoint);
        frameBatch.push(fallbackPt);
        this.lastSavedPoint = fallbackPt;}

    return frameBatch;}

/**
 * Advances the simulation for discrete maps.
 * Does the same thing, as a 'step' function, just without rk4 integrator,
 * because discrete maps use step-by-step formulas.
 */
public stepMap(
    lastPoint: StateVector,
    speed: number,
    nextStateFn: (state: StateVector) => StateVector,
): StateVector[] {
    // For maps, speed decides how many steps to calculate this frame
    const iterations = Math.max(1, Math.floor(speed * 10));
    const frameBatch: StateVector[] = [];
    let currentPoint = lastPoint;
    for (let i = 0; i < iterations; i++) {
        const nextPoint = nextStateFn(currentPoint);
        if (!SimulationValidator.isValidPoint(nextPoint as any)) {
            break;}
        currentPoint = nextPoint;
        frameBatch.push(Array.from(currentPoint));}
    return frameBatch;}

/** Generates a large batch of points all at once to draw a static shape
    on screen. */
public computeMapBatch(

```

```

    count: number,
    startPoint: StateVector,
    nextStateFn: (state: StateVector) => StateVector,): StateVector[] {
    const batch: StateVector[] = [...startPoint];
    let currentPoint = startPoint;
    for (let i = 0; i < count; i++) {
        const nextPoint = nextStateFn(currentPoint);
        if (!SimulationValidator.isValidPoint(nextPoint as any)) {
            break;}
        currentPoint = nextPoint;
        batch.push(Array.from(currentPoint));
    }
    return batch;}

public clear(): void {
    this.lastSavedPoint = null;}

public getDimension(): number {
    return this.scratch.y2.length;}}

```

ДОДАТОК Г

Реалізація допоміжного перехоплювача у програмі

```
interface UseSimulationLoopProps {
  side: Side;}

/**
 * Custom hook that runs math equations over and over in the background
 * using React Three Fiber's endless frame loop.
 */
export const useSimulationLoop = ({ side }: UseSimulationLoopProps) => {
  const addPoints = useSimulationStore((state) => state.addPoints);
  const setPoints = useSimulationStore((state) => state.setPoints);
  const      systemType      =      useSimulationStore((state)      =>
state.sims[side].systemType);
  const params = useSimulationStore((state) => state.sims[side].params);
  const system = useMemo(() => {
    return  SYSTEM_REGISTRY[systemType]  ||  SYSTEM_REGISTRY["lorenz"];},
[state.sims[side].systemType]);
  const stateDimension = useMemo(() => {
    return system.math.initialState?.length || 3;},
[state]);
  /** Keeps a single, persistent copy of math engine alive across renders.
  */
  const engineRef = useRef<SimulationEngine | null>(null);
  if  (!engineRef.current  ||  engineRef.current.getDimension()  !==
stateDimension) {
    engineRef.current = new SimulationEngine(stateDimension);}
  /** Gets the formula for ODEs. */
  const derivative = useMemo(() => {
    if (system.math.type !== "ode") return null;
    return system.math.getDerivative?.(params);},
[state, params]);
  /** Gets the formula for maps. */
  const nextStateFn = useMemo(() => {
    if (system.math.type !== "map") return null;
```

```

    return system.math.getNextState?.(params);
  }, [system, params]);
/** Static computation for Maps when parameters change. */
useEffect(() => {
  if (system.math.type === "map" && nextStateFn && engineRef.current){
const initial = system.math.initialState || [0.1, 0.1, 0];
    const batch = engineRef.current.computeMapBatch(5000, initial,
nextStateFn);
    setPoints(side, batch);}},
  [system, nextStateFn, side, setPoints]);
/**
  * If the user clicks 'Restart', wipe out the math engine's memory to
  start over.
  */
  const storePoints = useSimulationStore((state) =>
state.sims[side].points);
  useEffect(() => {
    if (storePoints.length <= 1 && engineRef.current && system.math.type
=== "ode") {
      engineRef.current.clear();}},
    [storePoints, system, side]);
  /** Runs on every frame to add draeing points. */
  useFrame((_state, delta) => {
    const realTimeSim = useSimulationStore.getState().sims[side];
    if (!realTimeSim) return;
    const { isPaused, speed, points: currentStorePoints } = realTimeSim;
    if (isPaused || !engineRef.current || currentStorePoints.length === 0)
return;
    const lastPoint = currentStorePoints[currentStorePoints.length - 1];
    if (system.math.type === "ode" && derivative) {
      const newBatch = engineRef.current.step(lastPoint, delta, speed,
derivative);
      if (newBatch.length > 0) {
        addPoints(side, newBatch);}}});
  const sim = useSimulationStore((state) => state.sims[side]);
  return { sim };};

```

ДОДАТОК Г

Схема даних Prisma

```

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
}

model User {
  id          String      @id @default(uuid())
  username    String      @unique
  email       String      @unique
  passwordHash String
  resetPasswordToken String? @unique
  resetPasswordExpires DateTime?
  createdAt   DateTime    @default(now())
  // Relation to allow user create many presets
  presets     Preset[]
  @@map("users")
}

model Preset {
  id          Int          @id @default(autoincrement())
  name        String
  systemType  String       // for example: "lorenz", "rossler", etc...
  // Allows save math parameters, example: { "sigma": 10, "rho": 28, "beta": 2.66 }
  parameters  Json
  // Save the camera for identical view
  cameraConfig Json?

```

```

// Save colors, gradients, etc.
visuals      Json?
isPublic     Boolean @default(true)
createdAt    DateTime @default(now())
userId       String
user         User     @relation(fields: [userId], references: [id],
onDelete: Cascade)
@@index([systemType])
@@map("presets")
}

```